

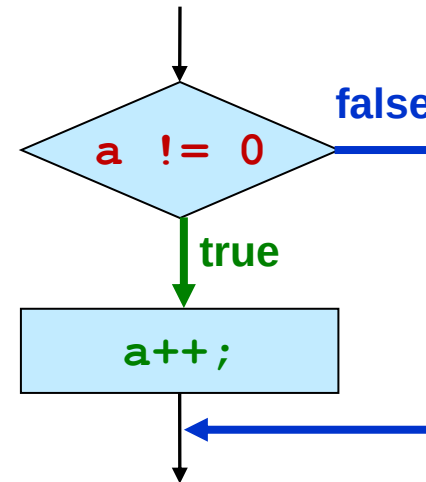
Branch Instructions

Computer Engineering 1

Motivation

- Branches may change the pc
 - "Non-linear" execution of programs
 - Taking decisions

```
uint32_t a;  
  
...  
if (a != 0) {  
    a++;  
}  
...
```



Agenda

- **Overview Branch Instructions** ¹⁾
- **Unconditional Branches**
 - `b` → direct, relative
 - `bx` → indirect, absolute
- **Conditional Branches**
 - Flag dependent branches
 - Arithmetic branches
 - signed vs. unsigned
- **Compare and Test**
 - `cmp` and `cmn`
 - `tst`

¹⁾ Subroutine calls will be discussed later

Learning Objectives

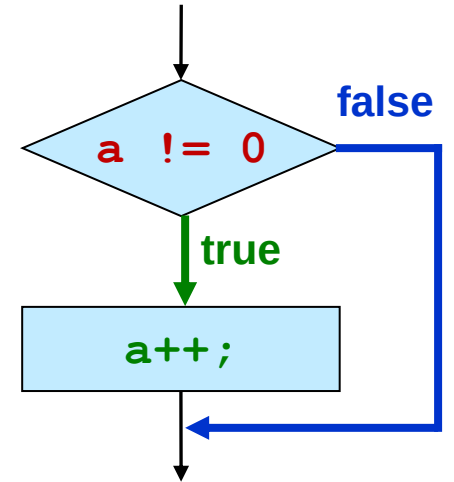
At the end of this lesson you will be able

- to explain what branch instructions are and how they work
- to classify a given branch instruction with regard to
 - conditional / unconditional
 - relative / absolute
 - direct / indirect
- to apply and discuss the different branch instructions
- to determine based on the settings of the flags whether a conditional branch is taken or not
- to distinguish, apply and explain the instructions **cmp**, **cmn** and **tst**

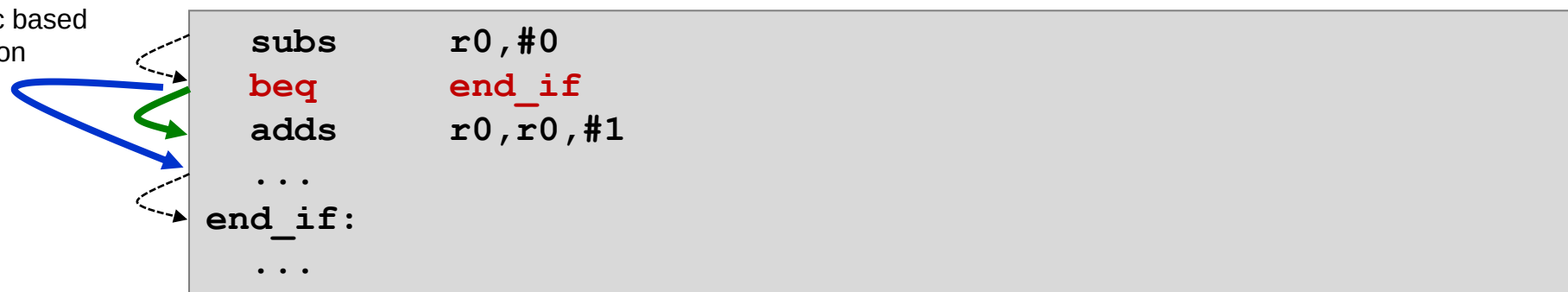
Motivation

- Branches may change the pc
 - "Non-linear" execution of programs
 - Taking decisions

```
uint32_t a;  
  
...  
if (a != 0) {  
    a++;  
}  
...
```



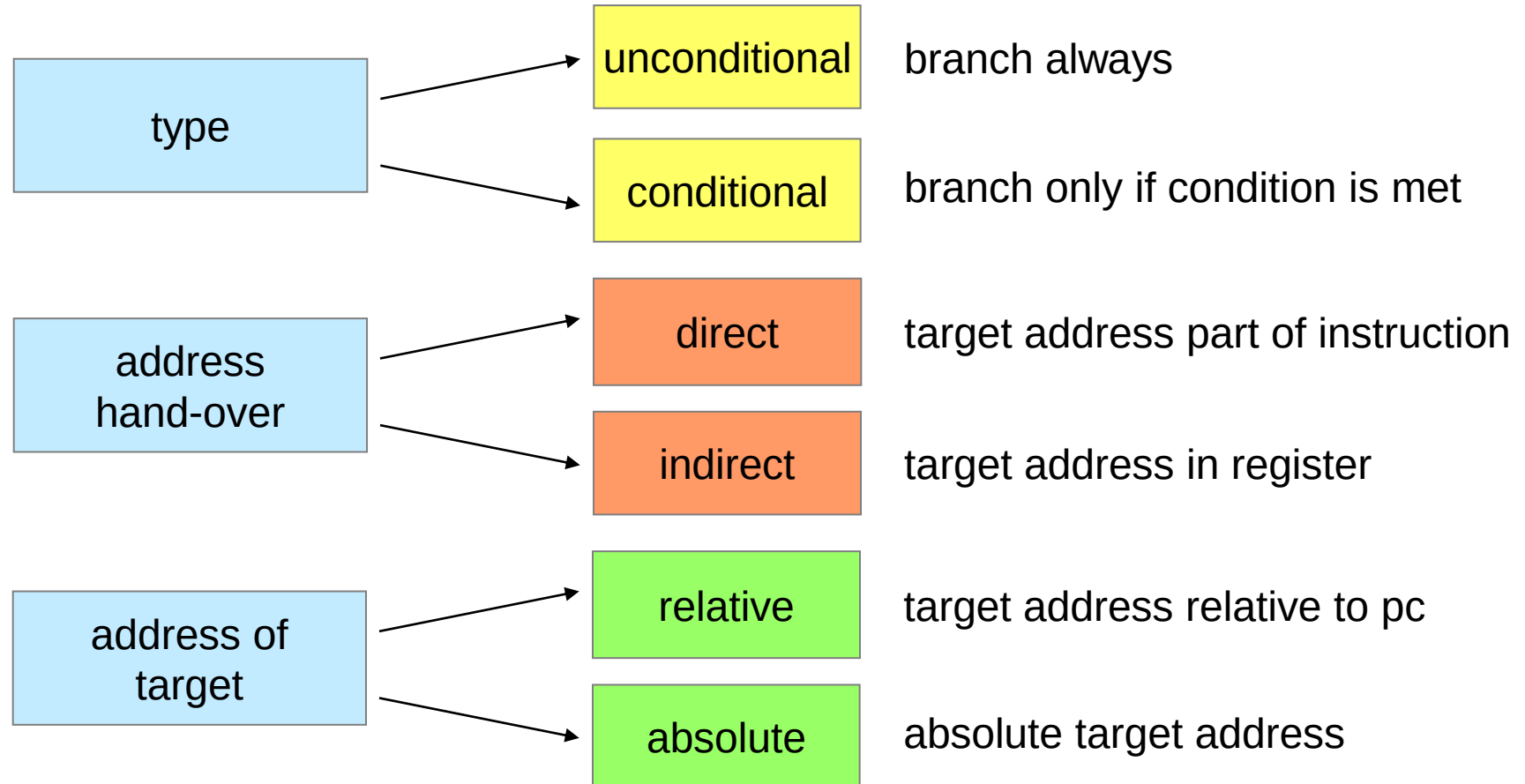
modify pc based
on decision



Note: In fact the compiler will use cmp instead of subs

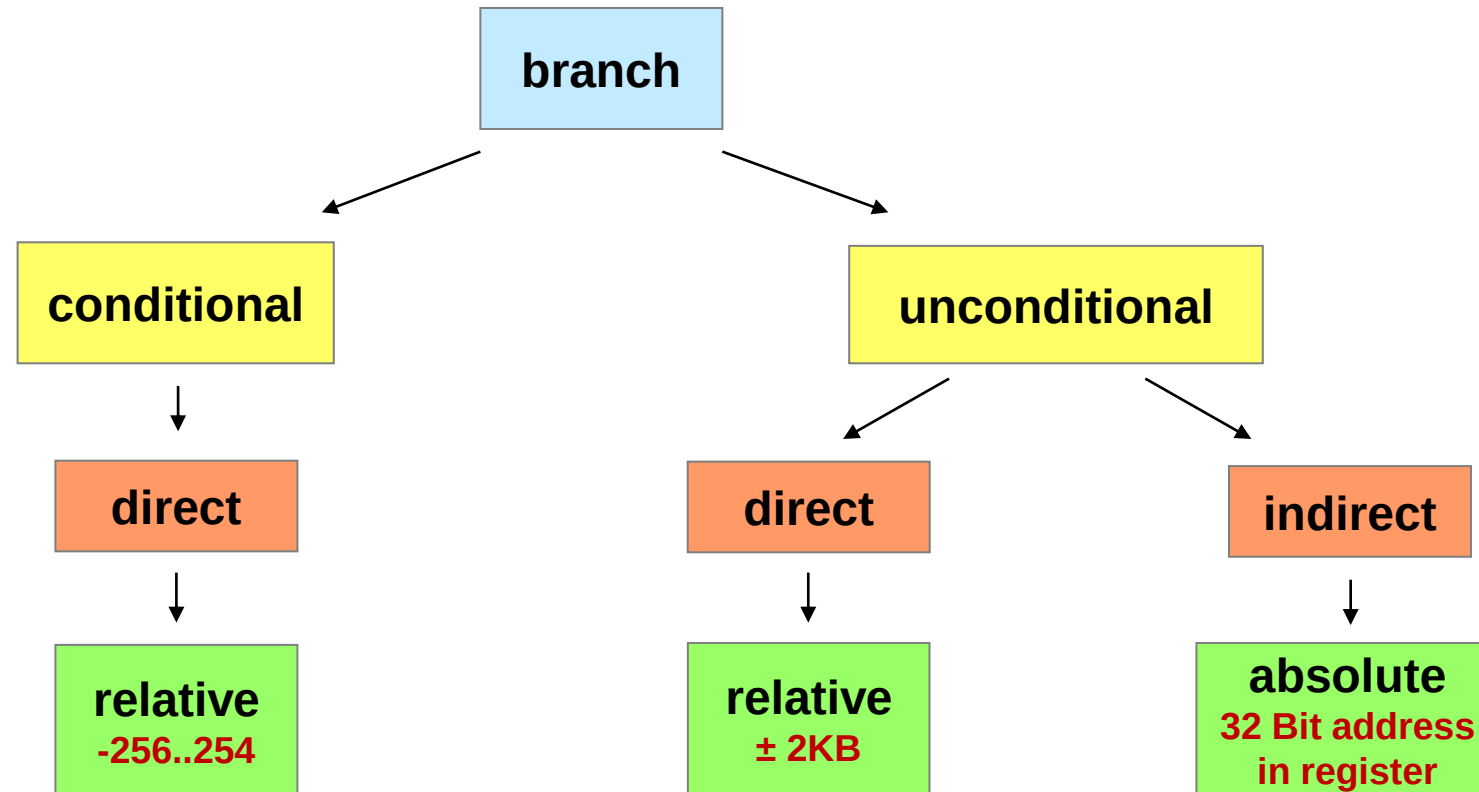
Overview Branch Instructions

Properties



Overview Branch Instructions

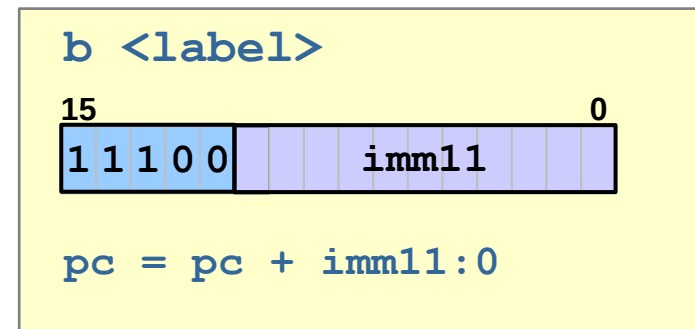
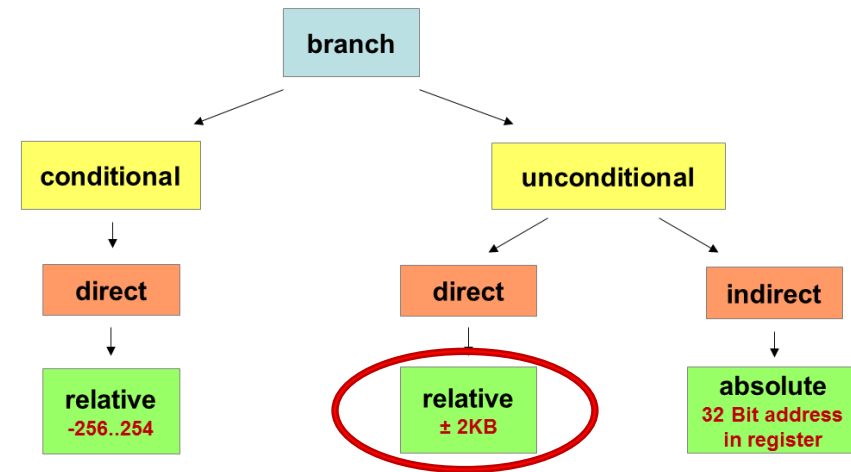
Overview ARMv6-M (Cortex-M0)



Unconditional Branches

Direct, Relative

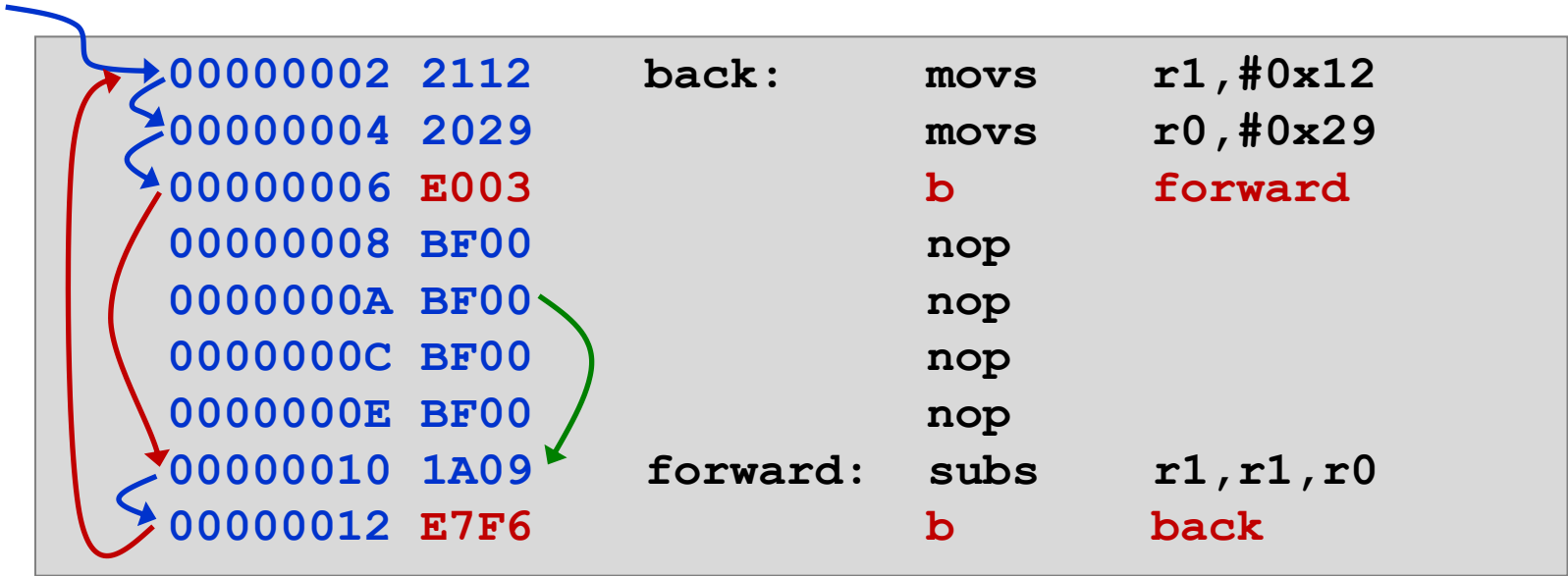
- b (immediate)
 - Unconditional
 - Direct
 - Relative (to pc)
 - imm11:0
 - Offsets from -2048d to +2046d



Unconditional Branches

... Direct, Relative

- Direct, relative branch → **b label**



1) Because of pipeline, pc is always **current address plus 4**

Forward

$$\begin{array}{rcl}
 & 0x0000'0006 & \text{Address of current instruction} \\
 + & 0x0000'0004 & \text{1)} \\
 + & \underline{0x0000'0006} & (0x003 \ll 1) = 0x006 \\
 & 0x0000'0010 &
 \end{array}$$

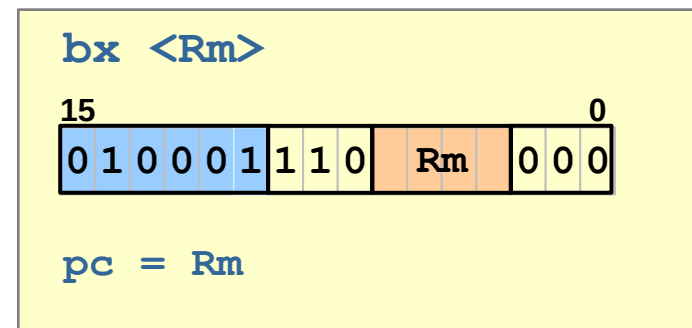
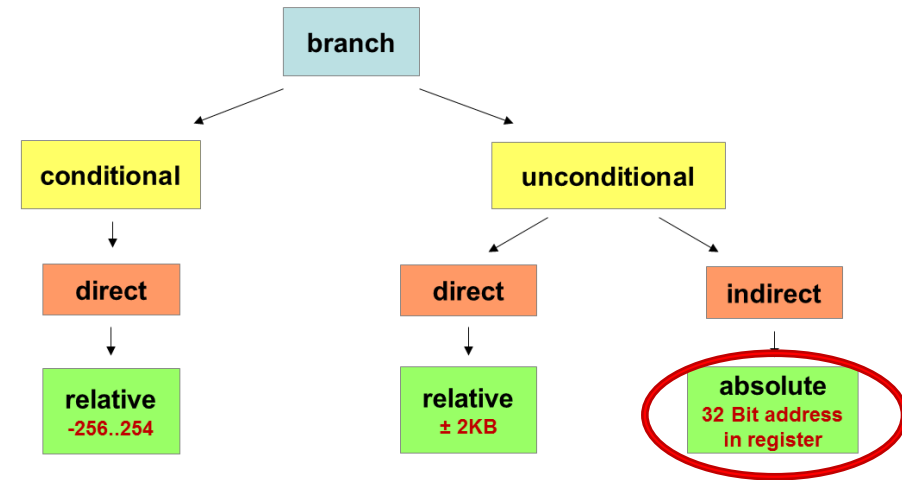
Back

$$\begin{array}{rcl}
 & 0x0000'0012 & \text{Address of current instruction} \\
 + & 0x0000'0004 & \text{1)} \\
 + & \underline{0xFFFF'FFEC} & (0x7F6 \ll 1) = 0xFEC \\
 & 0x0000'0002 & = -20d \\
 & & \text{sign-extended}
 \end{array}$$

Unconditional Branches

Indirect, Absolute

- bx
 - Branch and Exchange¹
 - Register Rm holds target address
 - Unconditional
 - Indirect
 - Absolute

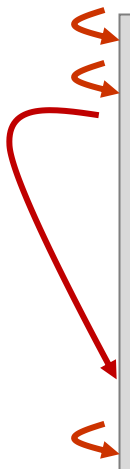


¹ Exchange: change execution mode, see next slides

Unconditional Branches

... Indirect, Absolute

- Indirect, absolute branch → **bx r0**



```
ldr    r0,=jmpaddr
bx     r0
@ following two pseudo instructions are necessary
@ for jumping to a label using bx or blx2:
.thumb           @ following code is Thumb-code
.thumb_func      @ next symbol (label) is Thumb-destination
jmpaddr:
adds    r0,r0,#0x13
...
```

- jmpaddr = 0x0000'001C → r0
- r0 → pc

¹explanation: see next slide

² blx is handled in the lecture „Control Structures“

bx: Branch and Exchange

Change Execution Mode ARM <-> Thumb

- Two execution modes for ARM processors: ARM and Thumb
- Switching between modes is controlled by bit 0 of a jump address:
 - Bit 0 = 0 → ARM code
 - Bit 0 = 1 → Thumb code
- Example: jump to address 0x1000
 - Address 0x1000 → ARM code
 - Address 0x1001 → Thumb code (same destination 0x1000, but different mode)
 - 0x1001 is not a real address: CPU ignores bit 0 for alignment, but uses it for mode selection
- Using **bx** or **blx**¹ can change the mode
 - Solution: use **.thumb** and **.thumb_func** to define mode of code and symbols

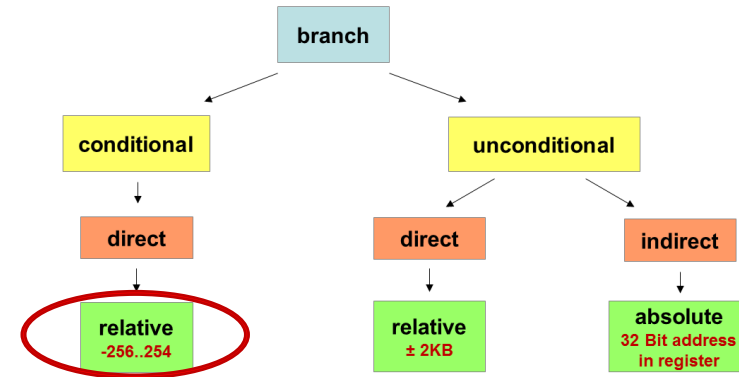
```
ldr    r0,=jmpaddr
bx     r0

.thumb
.thumb_func
jmpaddr:
adds   r0,r0,#0x13
...
```

¹ blx will be introduced in the lecture „Control Structures“

Conditional Branches

- **Flag-dependent** branches
 - Based on **one specific flag**
- **Arithmetic** branches
 - Based on one or more flags
 - e.g. after an arithmetic instruction
 - **unsigned** operations
 - **higher and lower**
 - **signed** operations
 - **greater and less**
- **Conditional branches are always relative on ARM**



Conditional Branches

Flag-dependent

Symbol	Condition	Flag
eq	Equal	Z == 1
ne	Not equal	Z == 0
cd	Carry set	C == 1
cc	Carry clear	C == 0
mi	Minus/negative	N == 1
pl	Plus/positive or zero	N == 0
vs	Overflow	V == 1
vc	No overflow	V == 0

source: Joseph Yiu: The definite Guide to the ARM Cortex M3, Page 63

Conditional Branches

Arithmetic - unsigned

- higher and lower

Symbol	Condition	Flag
eq	Equal	Z == 1
ne	Not equal	Z == 0
hs (=cs)	Unsigned higher or same	C == 1
lo (=cc)	Unsigned lower	C == 0
hi	Unsigned higher	C == 1 and Z == 0
ls	Unsigned lower or same	C == 0 or Z == 1

source: Joseph Yiu: The definite Guide to the ARM Cortex M3, Page 63

Conditional Branches

Arithmetic - signed

- greater and less

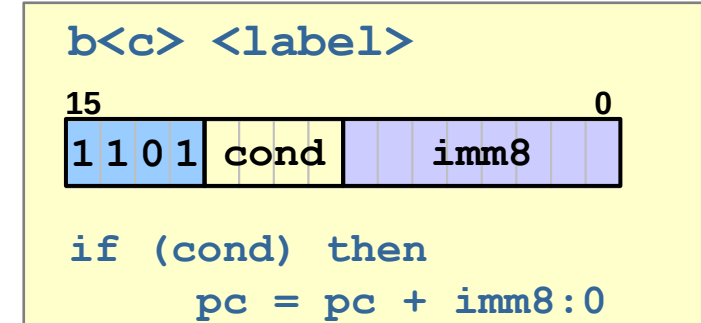
Symbol	Condition	Flag
eq	Equal	Z == 1
ne	Not equal	Z == 0
mi	Minus/negative	N == 1
pl	Plus/positive or zero	N == 0
vs	Overflow	V == 1
vc	No overflow	V == 0
ge	Signed greater than or equal	N == V
lt	Signed less than	N != V
gt	Signed greater than	Z == 0 and N == V
le	Signed less than or equal	Z == 1 or N != V

source: Joseph Yiu: The definite Guide to the ARM Cortex M3, Page 63

Conditional Branches

Opcodes

- imm8:0
 - Offset from -256d to +254d

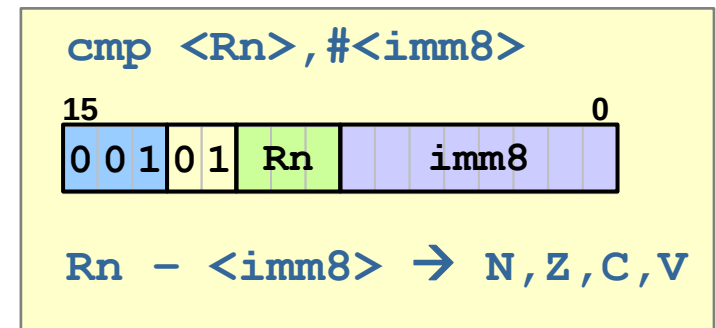
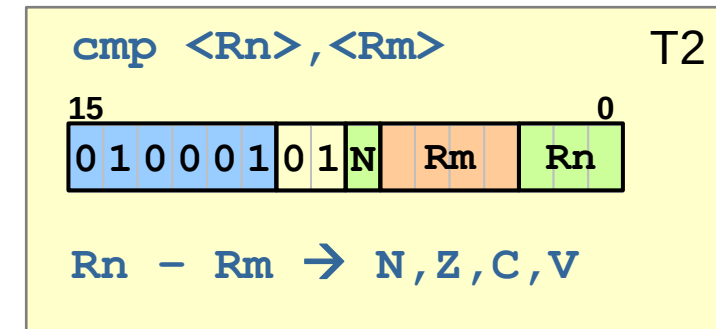
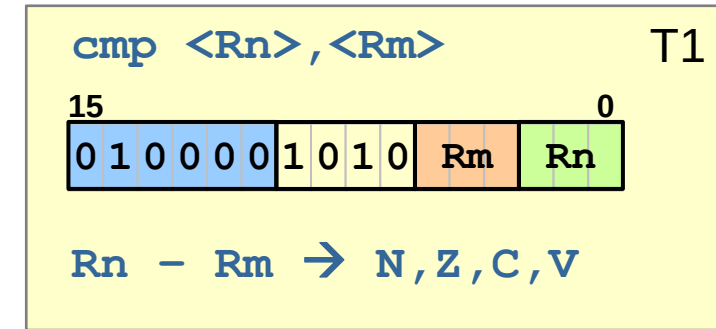


cond	short	Flag
0000	eq	Z == 1
0001	ne	Z == 0
0010	cs/hs	C == 1
0011	cc/lo	C == 0
0100	mi	N == 1
0101	pl	N == 0
0110	vs	V == 1
0111	vc	V == 0

cond	short	Flag
1000	hi	C == 1 and Z == 0
1001	ls	C == 0 or Z == 1
1010	ge	N == V
1011	lt	N != V
1100	gt	Z == 0 and N == V
1101	le	Z == 1 or N != V
1110	al	always
1111	--	--

Compare and Test cmp

- Same as subs, but without storing a result!
- Compare 2 operands
 - Higher/lower?
 - Greater/less?
 - Equal?
- **Only flags are affected!**
- Registers unchanged
- T2 also higher registers



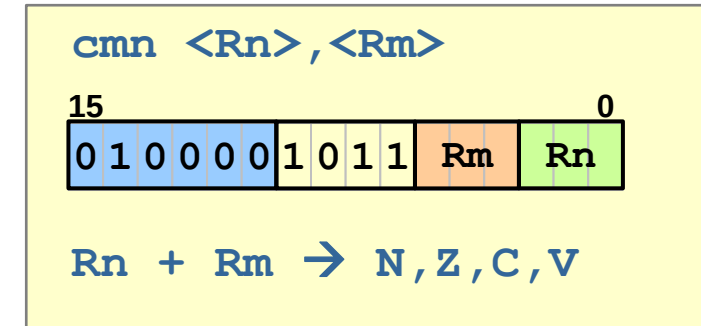
Compare and Test cmp

- cmp
 - cmp does not change registers
 - Example

```
cmp    r0, r1    @ r0 > r1 ?
bhi    go_on     @ if higher -> go_on
movs   r2, r1    @ otherwise exchange regs
movs   r1, r0
movs   r0, r2
go_on:
movs   r3, #5
...
```

Compare and Test cmn

- Same as adds, but without storing result!
- Compare 2 operands negative
- Only flags are affected!
- Registers unchanged
- Read cmn as
 - Is content of Rm equal to 2's complement of Rn?

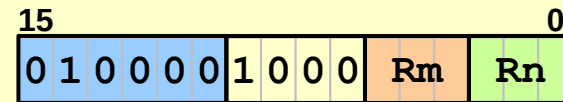


```
cmn    r0, r1    @ r0 equal minus r1?  
beq    next
```

Compare and Test tst

- Is a specific bit set?
- Logical AND without storing result
- Registers unchanged
- Changes only flags N and Z
- C and V unchanged

`tst <Rn>, <Rm>`



$Rn \ \& \ Rm \rightarrow N, Z$

```
.equ SWITCH_ADDRESS, 0x60000200
.equ S3_MASK, 0x00000008
ldr    r1,=SWITCH_ADDRESS
ldr    r0,[r1]           @ read switch data
ldr    r2,=S3_MASK
tst    r0,r2             @ bit S3 = 1 ?
bne    s3_equal_one      @ branch if Z = 0
s3_equal_zero:
...
s3_equal_one:
...
```

Exercise

– Which branches are taken?

Instruction	Z	C	N	V	“Taken” / “Not Taken”?
bne label	1	0	0	0	
blo label	0	0	0	0	
bhi label	0	1	0	0	
blt label	0	0	1	1	
ble label	1	0	1	1	

“Taken”

Execution is continued at the indicated label

“Not Taken”

Execution is continued at the instruction following the branch instruction

Summary

- **Branch Instructions Change **pc****
 - „Decision Making“ and Control Flow
- **Branch Instructions**
 - unconditional, relative, direct **b** $pc = pc \pm 2KB$
 - unconditional, absolute, indirect **bx** $pc = Rm$
 - conditional, relative, direct **Bxx** $pc = pc-256; pc+254$
- **Compare and Test**
 - **cmp**, **cmn** → **subs**, **adds** without result, but flags are set!
 - **tst** → **AND** without result, but flags are set!